

实验名称： 机器学习实践： 第二次实验报告

实验日期： 2026 年 6 月 28 日

实验地点： C4-519 机房

班级： 人工智能 24-2 班

姓名： 王晨枫

学号： 2024015937

一、实验内容

任务 2：卷积神经网络

一、实验内容

本次实验围绕卷积神经网络在图像分类任务中的应用展开，共包含两个相对独立的实验流程：MNIST 手写数字识别实验与 CIFAR-10 彩色图像分类实验。报告按照“先完整介绍 MNIST，再完整介绍 CIFAR-10”的顺序组织，避免两部分内容交叉穿插。

任务	数据集	主要内容	模型数量
任务一	MNIST	完成 0-9 手写数字分类, 并使用外部手写数字 7 图片进行推理验证	2 个
任务二	CIFAR-10	训练 base_cnn、wide_cnn、deep_cnn 三种结构, 对比性能并分析拟合状态	3 个

二、实验过程描述及结果记录

1. 实验环境与通用配置

本实验在 macOS 与 Conda 环境下完成。为避免系统默认 pyenv Python 缺少 lzma 模块导致 torchvision 导入失败，正式训练统一使用 Conda 的 ai 环境。训练设备自动选择 Apple MPS 后端，从而利用 Apple Silicon 的 Metal 加速能力。

项目	配置
操作系统	macOS 26.3.1 (arm64)
处理器架构	Apple Silicon (M 系列)
Conda 环境	ai

Python	3.10.20
Python 路径	/opt/miniconda3/envs/ai/bin/python
PyTorch / torchvision	2.11.0 / 0.26.0
NumPy / pandas / matplotlib	2.2.6 / 2.3.3 / 3.10.9
tqdm / Pillow	4.67.3 / 12.2.0
CUDA / Apple MPS	CUDA 不可用；Apple MPS 可用，正式训练装置为 mps
开发工具	VSCode / Cursor

文件/目录	功能说明
src/models.py	定义 CNN 模型结构与 build_model() 函数
src/train.py	完成数据加载、训练、测试、模型保存与 CSV 日志记录
src/plot_results.py	读取训练日志，生成 loss / accuracy 曲线图
src/predict_image.py	对外部图片进行预处理并载入模型推理
run_experiment.py / run_all_experiments.py	一键执行完整实验流程
outputs/logs、outputs/models、 outputs/figures	分别保存训练日志、模型权重和结果图表

2. MNIST 手写数字识别实验

MNIST 实验的目标是完成 0-9 手写数字识别，并进一步验证训练后模型对外部手写数字图片的推理能力。实验整体分为训练评估主线和图片推理主线：训练阶段先读取 MNIST 数据集，对灰度图像进行 Tensor 转换和标准化处理，再分别输入 SimpleCNN 与 BetterCNN 进行训练；测试阶段在完整测试集上记录 loss 与 accuracy；推理阶段使用训练效果更好的 BetterCNN 对手写数字 7 图片进行预处理和预测。

流程阶段	具体操作	实验产出
数据输入	读取 MNIST 训练集和测试集，输入为 $1 \times 28 \times 28$ 灰度图像。	获得训练样本、测试样本及标签。
数据预处理	执行 ToTensor 与 Normalize((0.1307,), (0.3081,)), 保证训练和推理输入分布一致。	得到标准化后的模型输入张量。
模型构建	分别构建 SimpleCNN 与 BetterCNN, 形成基础模型和改进模型对照。	获得两个可训练 CNN 模型。
训练与评估	使用 Adam 优化器和交叉熵损失函数训练 10 个 epoch, 每轮记录 train_loss、	生成训练日志、模型权重和曲线图。

	train_acc、test_loss、test_acc。	
图片推理	对手写数字 7 图片进行灰度化、裁剪、缩放、归一化，再载入 BetterCNN 推理。	输出预测类别 7 和置信度。

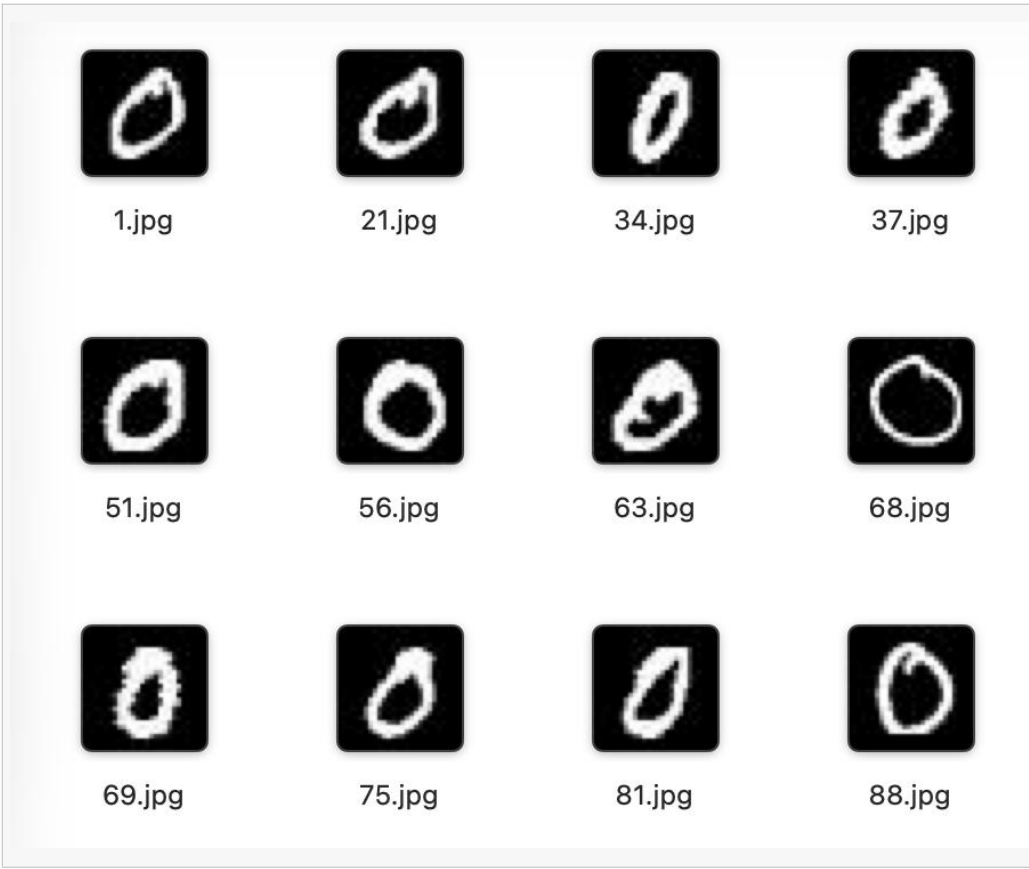


该架构图应体现两条路径：第一条是模型训练路径，用于说明数据如何进入两个 CNN 模型并生成训练指标；第二条是图片推理路径，用于说明外部手写数字图片如何经过预处理后进入已训练模型。通过这种结构，可以看出本实验不仅完成了标准数据集分类，也验证了模型在简单外部输入上的应用效果。

2.2 数据集与预处理

项目	数值
数据集	MNIST
训练集数量	60000
测试集数量	10000
类别数	10 类，数字 0 到 9
图片大小	1 × 28 × 28
训练集 shuffle	是
测试集 shuffle	否

MNIST 图片为单通道灰度图，模型输入尺寸为 $1 \times 28 \times 28$ 。预处理采用 `ToTensor()` 将图片转为张量，再使用 `Normalize((0.1307,), (0.3081,))` 进行标准化。训练和外部图片推理采用相同归一化参数，以保证输入分布一致。



图表说明：该图用于展示 MNIST 数据集样例。样例图应体现 MNIST 图像具有黑底、白色手写数字、灰度图、数字居中等特征，为后续外部图片预处理提供对照标准。

2.3 模型结构设计

模型	结构特点	参数量	设计目的
SimpleCNN	Conv(1,16)+ReLU+MaxPool; Conv(16,32)+ReLU+MaxPool; Linear(32 × 7 × 7,128); Linear(128,10)	206922	基础 CNN 对照模型，验证简单卷积结构的有效性
BetterCNN	Conv(1,32)+BN+ReLU; Conv(32,64)+BN+ReLU+MaxPool; Dropout2d; Conv(64,128)+ReLU+MaxPool; Linear(128 × 7 × 7,256); Dropout; Linear(256,10)	1701322	通过更大通道数、BatchNorm 与 Dropout 提升泛化能力



图表说明：该结构图应突出 SimpleCNN 与 BetterCNN 的差异。SimpleCNN 层数少、参数量小，适合作为基线模型；BetterCNN 增加通道数并加入 BatchNorm、Dropout，目的是提高特征提取能力并缓解过拟合。

2.4 训练设置与运行命令

参数	取值
Epochs	10
Batch size	128
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Device	mps

```
cd mnist_task
MPLCONFIGDIR="$(pwd)/outputs/matplotlib" conda run -n ai python run_experiment.py --epochs 10
```

训练脚本实际依次训练 simple_cnn 与 better_cnn，并在训练完成后调用绘图脚本生成 train loss、test loss、train accuracy、test accuracy 和最终测试准确率对比图。

2.5 MNIST 结果图表与说明

模型	最终 Train Acc	最终 Test Acc	最高 Test Acc	最终 Test Loss	参数量
SimpleCNN	99.6583%	98.9000%	99.0000% (第 9 轮)	0.036420	206922

BetterCNN	99.3550%	99.3400%	99.4000% (第 9 轮)	0.021915	1701322
-----------	----------	----------	------------------	----------	---------

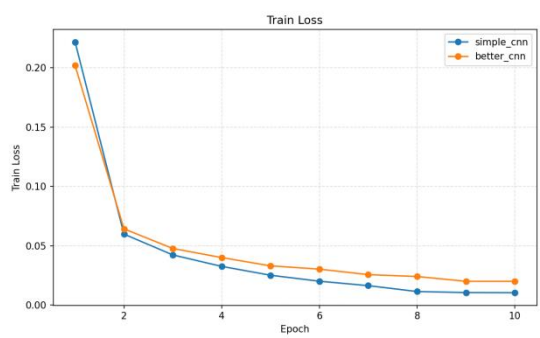


图 3 MNIST 训练损失曲线

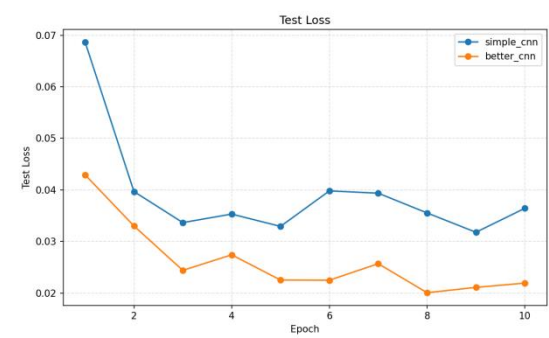


图 4 MNIST 测试损失曲线

图 3 显示两个模型的训练损失在前 2 个 epoch 内迅速下降，说明模型能够快速学习 MNIST 的基本数字特征。后期 SimpleCNN 的训练损失略低于 BetterCNN，主要因为 BetterCNN 中 Dropout 会在训练阶段随机抑制部分神经元，使训练损失不一定最低。图 4 中 BetterCNN 的测试损失整体低于 SimpleCNN，且后期波动更小，说明加强模型虽然参数更多，但在测试集上的分类信心和泛化能力更好。

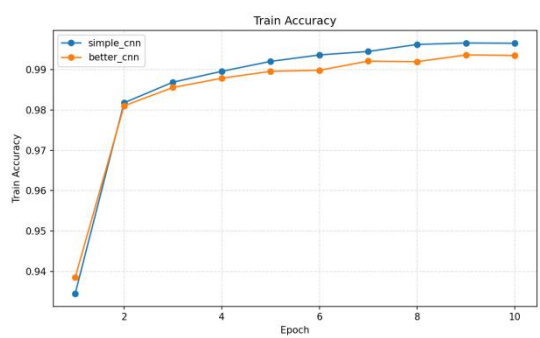


图 5 MNIST 训练准确率曲线

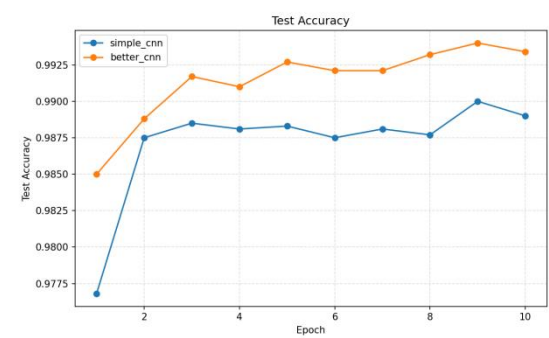


图 6 MNIST 测试准确率曲线

图 5 中两个模型的训练准确率均在第 2 轮后超过 98%，说明 MNIST 任务相对简单，卷积网络能够快速收敛。SimpleCNN 末期训练准确率略高，但并不代表其泛化更好。图 6 显示 BetterCNN 的测试准确率从训练初期开始就整体高于 SimpleCNN，最高达到 99.40%，说明 BatchNorm 和 Dropout 对提升测试集表现具有积极作用。

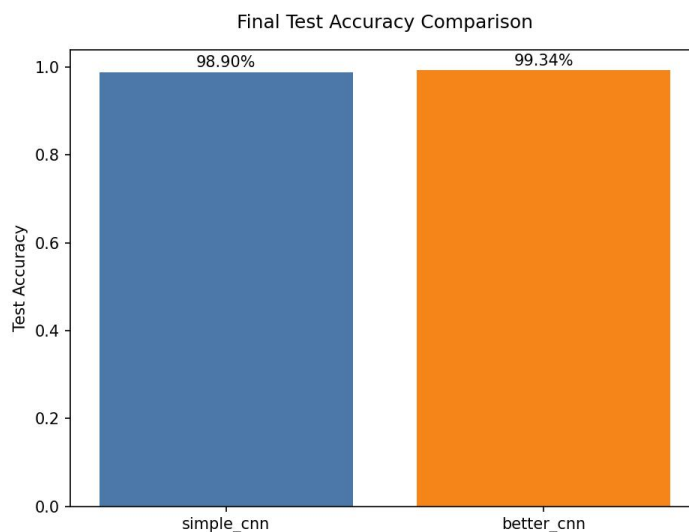


图 7 MNIST 两模型最终测试准确率对比

图 7 将最终测试准确率以柱状图形式展示。BetterCNN 最终测试准确率为 99.34%，比 SimpleCNN 的 98.90% 高 0.44 个百分点。由于 MNIST 两个模型的准确率都已经较高，该提升幅度虽然不大，但结合更低的测试 loss 可以说明 BetterCNN 的泛化效果更稳定。

2.6 MNIST 模型对比分析

SimpleCNN 作为基础模型，结构简单、参数量小，已经能够达到 98.90% 的测试准确率，说明两层卷积加池化的基本 CNN 结构足以提取 MNIST 手写数字的主要形状特征。其优势是计算开销较小，训练速度快，但测试 loss 高于 BetterCNN，说明分类置信度略弱。

BetterCNN 在结构上增加卷积通道数，并引入 BatchNorm 与 Dropout。虽然其参数量从 206922 增加到 1701322，但测试准确率提升到 99.34%，最高测试准确率达到 99.40%。更重要的是，BetterCNN 的最终测试 loss 为 0.021915，明显低于 SimpleCNN 的 0.036420，说明其对测试样本的判别更加稳定。

从拟合状态看，SimpleCNN 的训练准确率高于测试准确率，存在轻微过拟合，但由于测试准确率仍较高，该现象可以接受。BetterCNN 参数更多，但由于 BatchNorm 稳定中间特征分布，Dropout 抑制过度依赖局部特征，因此其测试集表现反而更好。

2.7 手写数字图片推理验证

为验证模型对外部输入图片的识别能力，实验使用数字 7 图片进行推理。推理脚本首先将外部图片转换为灰度图，再经过裁剪、缩放、反色、归一化等步骤，使其尽量接近 MNIST 数据集的输入格式。

项目	结果
测试图片	images/mnist_sample_7.png / 手写数字 7 图片
真实标签	7

使用模型	outputs/models/better_cnn.pth
预测结果	7
置信度	1.0000
预处理流程图	outputs/figures/mnist_sample_7_preprocess.png



图 8 外部手写数字 7 原始图片

图 8 展示了外部输入的手写数字 7。该图片与 MNIST 原图不同，通常表现为白底黑字、尺寸较大、存在背景区域，因此不能直接输入模型，需要进行预处理。

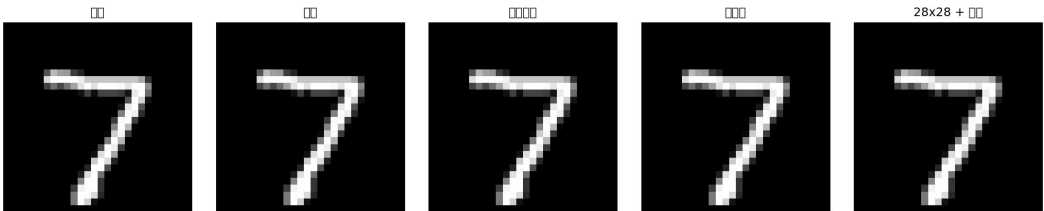


图 9 手写数字 7 预处理流程图

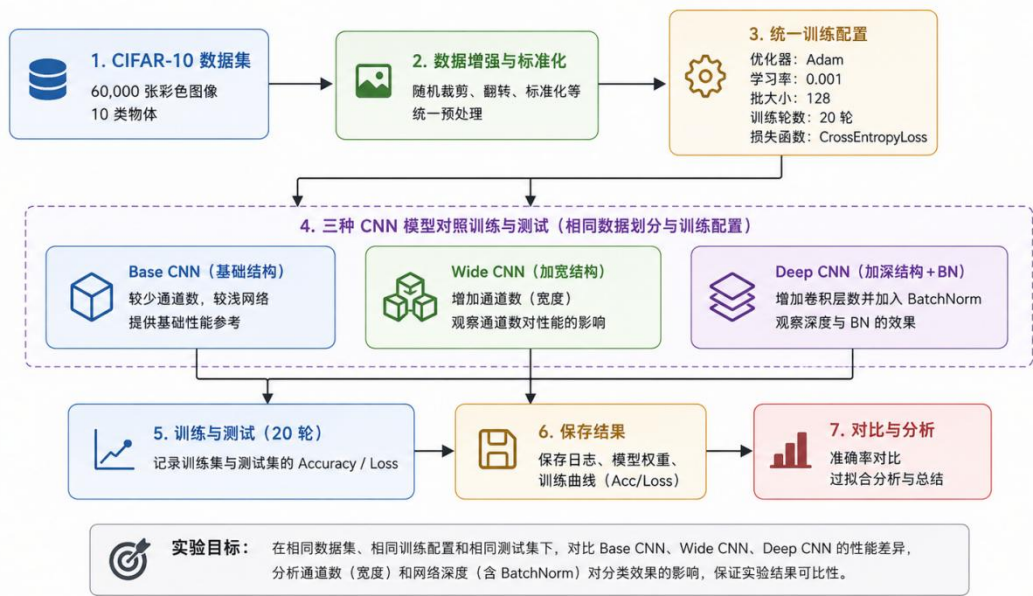
图 9 展示了从外部图片到模型输入的转变过程。预处理的核心目标是将白底黑字图片转换为与 MNIST 类似的黑底白字 28×28 灰度图，并保证数字位于图像中心。最终模型输出数字 7，置信度为 1.0000，说明 BetterCNN 不仅在标准测试集上表现较好，也能完成简单外部手写数字图片的推理验证。

3. CIFAR-10 彩色图像分类实验

CIFAR-10 实验的目标是比较三种不同 CNN 结构在彩色图像分类任务上的表现。实验整体按照“数据增强—模型对照—训练记录—结果可视化—拟合分析”的顺序展开。与 MNIST 相比，CIFAR-10 图像为 $32 \times 32 \times 3$ 彩色图像，类别差异更加复杂，因此本实验重点观察增加通道数、增

加网络深度以及引入 BatchNorm 后对模型性能的影响。		
流程阶段	具体操作	实验产出
数据输入	读取 CIFAR-10 训练集和测试集，类别包括 airplane、automobile、bird、cat、deer、dog、frog、horse、ship、truck。	获得 50000 张训练图像和 10000 张测试图像。
数据增强与标准化	训练集使用 RandomCrop、RandomHorizontalFlip、ToTensor、Normalize；测试集仅使用 ToTensor 和 Normalize。	增强模型对位置变化和翻转变化的适应能力。
模型构建	构建 base_cnn、wide_cnn、deep_cnn，分别对应基础、加宽、加深三种结构。	形成三组可对比模型。
训练与评估	每个模型训练 20 个 epoch，使用 Adam、CrossEntropyLoss 和 CosineAnnealingLR，并记录四项指标。	得到逐轮日志、最佳模型权重和测试结果。
结果对比	绘制每个模型 loss/accuracy 曲线、三模型 test_acc 对比图和各类别准确率表。	完成模型性能排序与类别差异分析。
拟合分析	结合 train-test 差距、loss 曲线和测试准确率变化判断过拟合/欠拟合。	得出 deep_cnn 综合表现最佳的结论。

CIFAR-10 实验整体架构流程图（三模型对照实验）



该架构图应重点突出“三模型对照实验”的思想。base_cnn 用于提供基础性能参考，wide_cnn 用于观察增加通道数带来的提升，deep_cnn 用于观察增加卷积层数并加入 BatchNorm 后的效果。最终通过统一训练配置和相同测试集评价，保证三种结构的结果具有可比性。

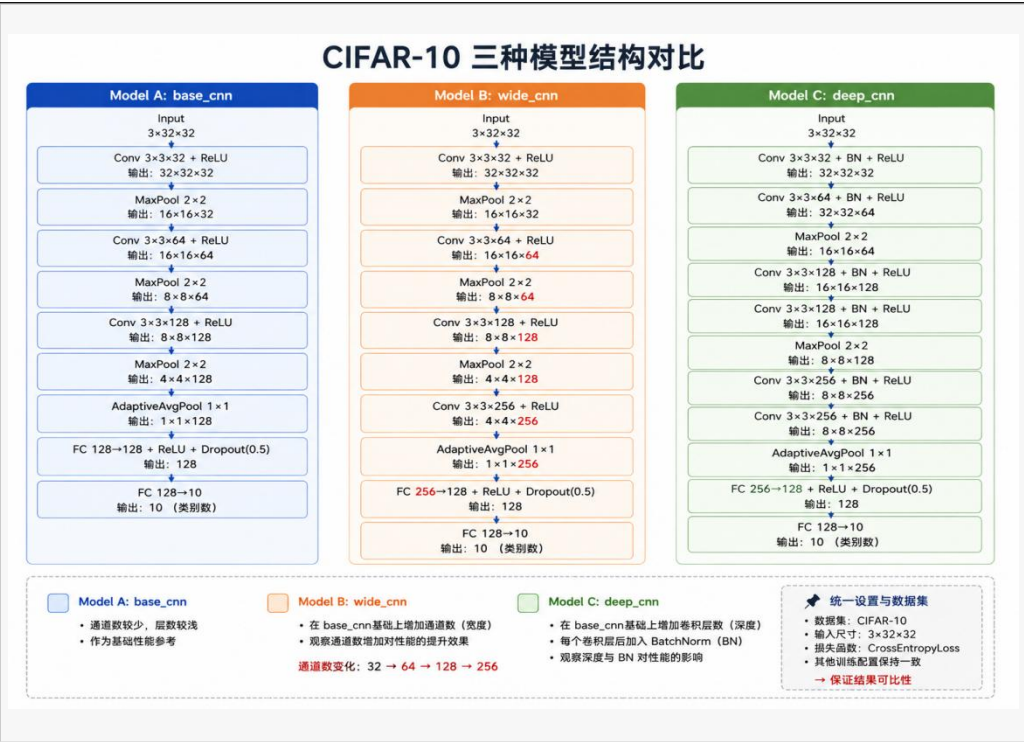
3.2 数据集与预处理

项目	数值
数据集	CIFAR-10
训练集数量	50000
测试集数量	10000
类别数	10 类
图片大小	32 × 32 × 3
batch_size	128
num_workers	2

CIFAR-10 为彩色图像分类数据集，类别包括 airplane、automobile、bird、cat、deer、dog、frog、horse、ship、truck。训练集采用 RandomCrop(32, padding=4)、RandomHorizontalFlip、ToTensor 和 Normalize 进行增强与标准化；测试集仅进行 ToTensor 与 Normalize，避免随机增强影响评估结果。

3.3 三种模型结构设计

模型	channels	stage_depths	BatchNorm	Dropout	结构说明
base_cnn	[32,64,128]	[1,1,1]	否	0.5	基础模型，共 3 层卷积，作为 baseline
wide_cnn	[64,128,256]	[1,1,1]	否	0.5	加宽模型，深度不变，通道数加倍
deep_cnn	[32,64,128]	[2,2,2]	是	0.5	加深模型，共 6 层卷积，并引入 BatchNorm



模型设计采用递进式对比: base_cnn 用于提供基础性能; wide_cnn 只增加通道数, 用来观察“加宽”是否有效; deep_cnn 增加每个 stage 的卷积层数并加入 BatchNorm, 用来观察“加深并稳定训练”是否能进一步提升性能。

3.4 训练设置与运行命令

参数	取值
Epochs	20
Batch size	128
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Scheduler	CosineAnnealingLR(T_max=20)
Device	auto → mps

conda activate ai

cd ml-task2-cnn

python run_all_experiments.py --epochs 20 --device auto

run_all_experiments.py 会依次训练三个模型, 并自动保存每轮训练日志、最佳模型权重、单模型曲线图、三模型对比图以及 experiment_summary.csv 汇总文件。

3.5 CIFAR-10 结果图表与说明

模型	最终 Train Acc	最终 Test Acc	最佳 Test Acc	最佳 Epoch	Train-Test 差距
base_cnn	71.51%	71.45%	71.51%	19	0.06%
wide_cnn	79.12%	77.34%	77.44%	19	1.78%
deep_cnn	89.46%	86.95%	86.95%	20	2.51%

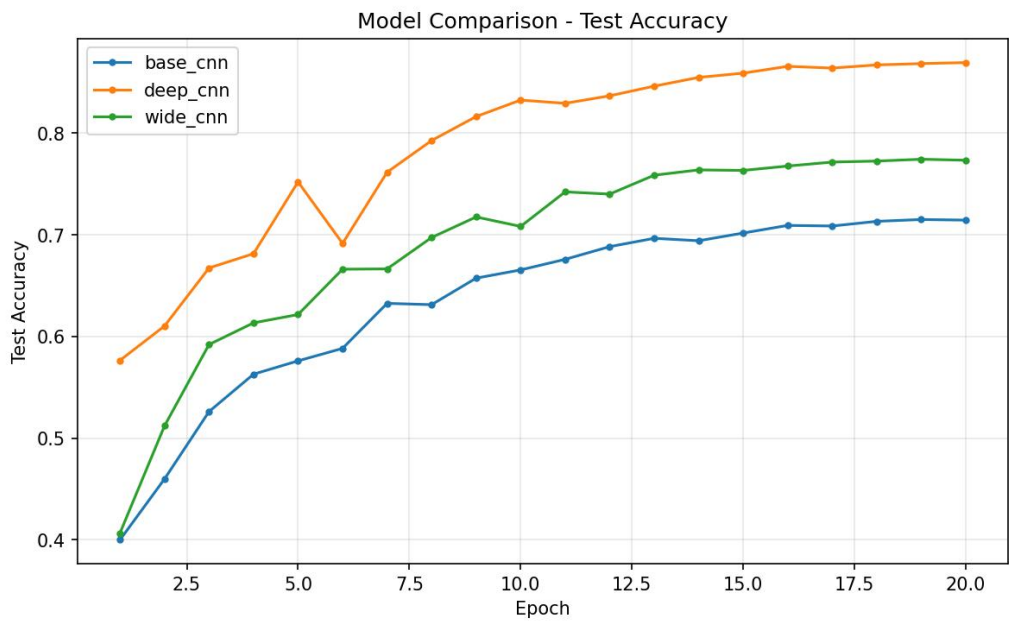


图 11 CIFAR-10 三模型测试准确率曲线对比

图 11 展示三种模型在 20 个 epoch 内的测试准确率变化。base_cnn 从约 40% 提升到 71.45%，说明基础 CNN 能够学习到一定图像特征，但性能上限较低。wide_cnn 在相同深度下提高通道数，最终测试准确率提升到 77.34%，说明加宽网络能够增强特征表示。deep_cnn 从第 1 轮开始整体领先，最终达到 86.95%，说明对于 CIFAR-10 这类复杂彩色图像任务，加深网络并使用 BatchNorm 的效果最明显。

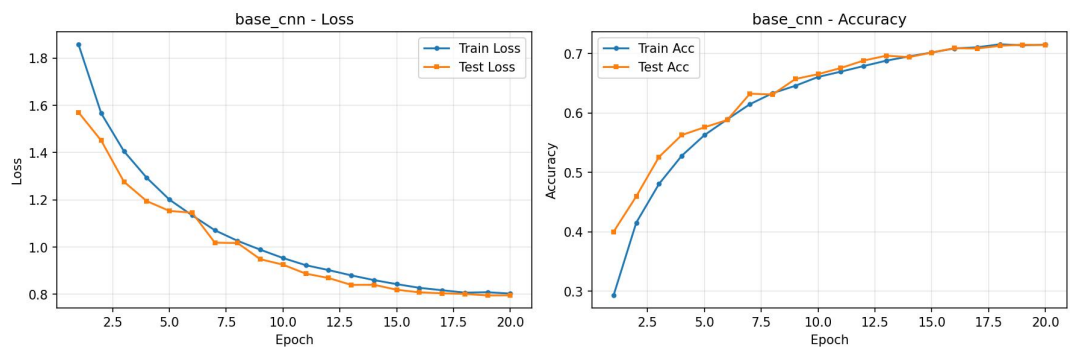


图 12 base_cnn loss/accuracy 曲线

图 12 中 base_cnn 的训练 loss 与测试 loss 同步下降，训练准确率与测试准确率非常接近，最终 train-test 差距仅 0.06%。这说明 base_cnn 没有明显过拟合，但由于模型只有 3 层卷积，特征提取能力

有限，最终准确率停留在 71% 左右，更接近“容量受限”的状态。

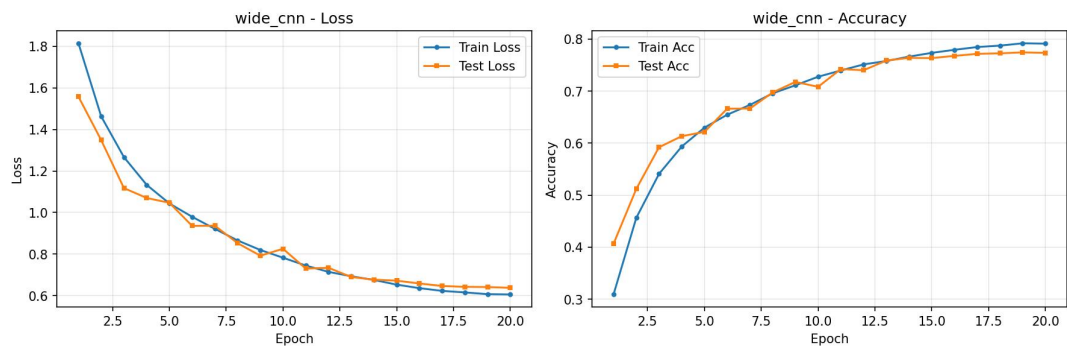


图 13 wide_cnn loss/accuracy 曲线

图 13 显示 wide_cnn 在多数 epoch 中训练与测试指标同步改善。相比 base_cnn，wide_cnn 的通道数加倍，使模型能够提取更多边缘、纹理和颜色组合特征，因此最佳测试准确率提升到 77.44%。但其最终 train-test 差距为 1.78%，比 base_cnn 略大，说明模型容量增加后出现轻微过拟合倾向。

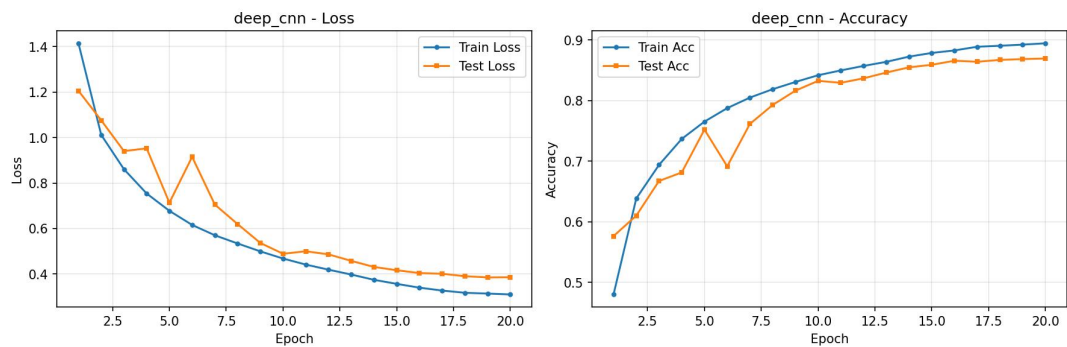


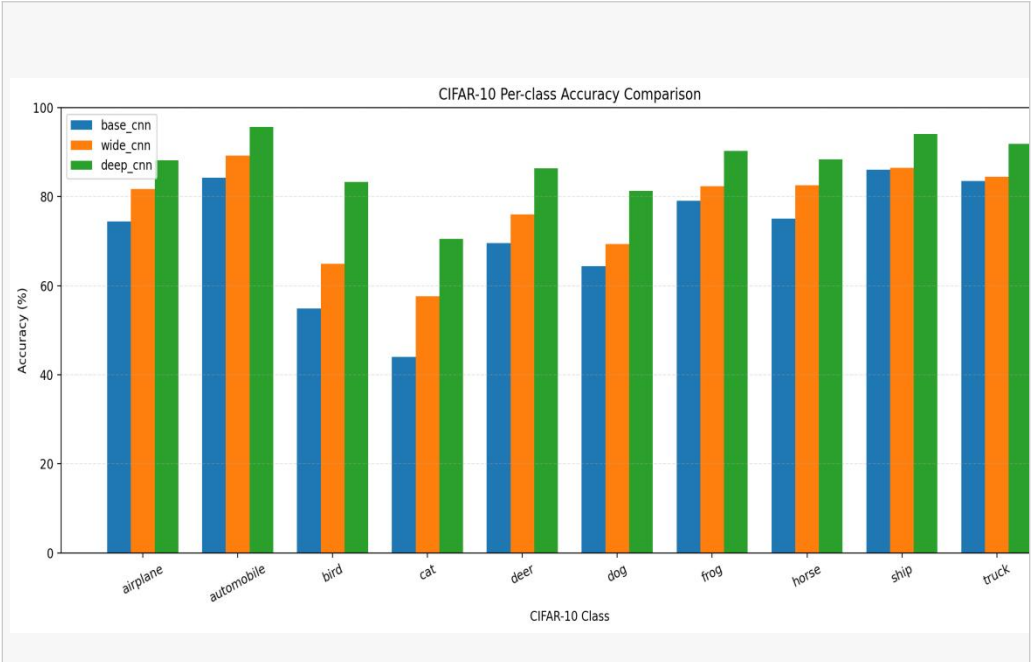
图 14 deep_cnn loss/accuracy 曲线

图 14 中 deep_cnn 的测试准确率在前期存在一定波动，尤其第 6 轮测试准确率出现回落，但随后持续提升，并在第 20 轮达到最高值 86.95%。这说明深层模型训练初期对参数和数据增强更敏感，但 BatchNorm 使训练后期趋于稳定。最终 deep_cnn 的性能明显高于 base_cnn 与 wide_cnn，说明“加深网络 + BatchNorm”的组合比单纯增加通道数更有效。

3.6 各类别准确率分析

类别	base_cnn	wide_cnn	deep_cnn
airplane	74.40%	81.70%	88.10%
automobile	84.20%	89.20%	95.60%
bird	54.90%	64.90%	83.30%
cat	44.00%	57.60%	70.50%
deer	69.60%	76.00%	86.30%
dog	64.40%	69.40%	81.30%
frog	79.10%	82.30%	90.20%

horse	75.00%	82.50%	88.30%
ship	86.00%	86.40%	94.10%
truck	83.50%	84.40%	91.80%



图表说明：建议将每类准确率绘制为分组柱状图。表中可以看出，automobile、ship、truck 等轮廓相对规则类别更容易识别，而 cat、bird、dog 等细粒度动物类别准确率较低，原因是这些类别在低分辨率图像中形状相似、背景复杂，容易相互混淆。deep_cnn 在所有类别上均优于另外两个模型，说明其综合特征提取能力最强。

3.7 CIFAR-10 模型对比与拟合分析

base_cnn 是基础 CNN 模型，最终 train_acc 为 71.51%，test_acc 为 71.45%，两者几乎一致，说明模型泛化稳定。但准确率较低，反映出基础模型在复杂彩色图像分类上的表达能力不足。

wide_cnn 保持深度不变，仅将通道数从 [32,64,128] 提升到 [64,128,256]。最佳测试准确率提升到 77.44%，相比 base_cnn 提高约 5.93 个百分点，说明增加通道数能够提升模型表示能力。但 train-test 差距扩大到 1.78%，说明模型容量增加也会带来一定过拟合倾向。

deep_cnn 将每个 stage 的卷积层数从 1 层增加到 2 层，共 6 层卷积，并引入 BatchNorm。它最终获得 86.95% 的测试准确率，明显优于其他两个模型。虽然其 train-test 差距为 2.51%，存在轻微过拟合，但测试准确率在第 20 轮仍达到最高，说明模型仍具有进一步训练和调参空间。

模型	拟合状态	分析结论
base_cnn	无明显过拟合，性能受限	train-test 差距仅 0.06%，但准确率较低，说明模型容量偏弱
wide_cnn	轻微过拟合	加宽提升性能，但 train-test 差距扩大到 1.78%

deep_cnn	轻微过拟合，综合最佳	train-test 差距 2.51%，但测试准确率最高，且第 20 轮仍在提升
----------	------------	--

4. 产出文件与结果保存

类别	文件示例	说明
训练日志	outputs/logs/*_history.csv / *_train_log.csv	保存每轮 train_loss、train_acc、test_loss、test_acc
模型权重	outputs/models/*.pth	保存训练后的模型参数或最佳权重
结果图表	outputs/figures/*.png	保存 loss / accuracy 曲线、模型对比图、预处理流程图
终端日志	outputs/logs/experiment_run.log	保存完整训练终端输出
汇总结果	outputs/logs/experiment_summary.csv	保存 CIFAR-10 三模型核心指标汇总

5. 实验总结

1. 本实验完整完成 MNIST 手写数字识别和 CIFAR-10 彩色图像分类两个 CNN 实践任务。
2. MNIST 实验中，SimpleCNN 已能达到 98.90% 的测试准确率，BetterCNN 通过增加通道数、BatchNorm 和 Dropout 将最终测试准确率提升到 99.34%。
3. CIFAR-10 实验中，base_cnn、wide_cnn、deep_cnn 的最佳测试准确率分别为 71.51%、77.44%、86.95%，说明模型结构对分类性能影响明显。
4. 加宽模型能够提升特征表示能力，但加深网络并引入 BatchNorm 的效果更显著。
5. 从过拟合和欠拟合分析看，各模型均未出现严重训练异常，deep_cnn 虽有轻微过拟合，但仍具有最佳综合表现。
6. 手写数字图片推理结果正确，说明训练后的模型不仅能完成标准测试集分类，也能用于简单外部图像识别。

总体而言，本实验完整体现了 CNN 图像分类实验的基本流程：数据处理、模型设计、训练评估、结果可视化和模型分析。实验结果表明，合理增加模型深度、使用 BatchNorm 和 Dropout 等结构改进，能够有效提升图像分类任务的性能。

6. 改进方向

虽然本实验已经完成了 MNIST 手写数字识别和 CIFAR-10 彩色图像分类两个任务，并通过不同 CNN 结构对模型效果进行了对比分析，但实验仍存在一定改进空间。

第一，在 MNIST 外部图片推理部分，本实验主要使用手写数字 7 作为验证样例，样本数量相对较少。虽然模型能够正确识别该图片，并输出较高置信度，但单个样例不能完全代表模型对所有外部手写数字图片的泛化能力。后续可以增加更多不同数字、不同书写风格、不同拍摄角度和不同光照条件下的手写图片进行测试，从而更全面地验证模型的实际应用效果。

第二，在 CIFAR-10 分类实验中，虽然 deep_cnn 模型取得了最高测试准确率，但从各类别

准确率可以看出，**cat**、**bird**、**dog** 等细粒度动物类别的识别效果仍然相对较弱。这说明模型在区分外观相似、背景复杂、形态变化较大的类别时仍存在不足。后续可以尝试更强的数据增强方法，例如随机颜色扰动、**Cutout**、**Mixup** 等，也可以引入更深的残差网络结构，如 **ResNet**，以进一步提升模型对复杂图像特征的提取能力。

第三，本实验主要使用训练集进行训练，并直接在测试集上记录模型性能。对于更加规范的机器学习实验流程，可以从训练集中额外划分验证集，用验证集进行模型选择、超参数调整和早停判断，最终只在测试集上进行一次最终评价。这样可以避免模型选择过程对测试集产生依赖，使实验结果更加客观、可靠。

综上，本实验已经较完整地展示了 **CNN** 图像分类的基本流程，但在外部样本验证数量、复杂类别识别能力和实验评估规范性方面仍有提升空间。后续如果继续优化，可以从增加测试样本、改进数据增强、引入更强网络结构和增加验证集机制等方面展开。

成 绩： _____

指导教师（签名）： _____

附录：两个实验核心源代码

CIFAR-10 三模型对比实验与 MNIST 手写数字识别实验

本附录整理两个实验的核心代码，用于在报告中说明模型定义、数据处理、训练执行、结果记录与图片推理全过程。正文报告主要展示实验过程和分析结果，本附录用于补充关键实现细节，便于复现实验。

附录内容概览

部分	内容	作用
附录 A	CIFAR-10 三模型对比实验	展示 base_cnn、wide_cnn、deep_cnn 的配置、模型结构、数据处理、训练评估和一键运行命令
附录 B	MNIST 手写数字识别实验	展示 SimpleCNN、BetterCNN 的模型定义、MNIST 数据加载、训练日志记录和实拍图片推理流程
附录 C	统一项目结构	说明两个项目的代码模块和输出目录含义
附录 D	运行命令汇总	给出可以直接复制到终端执行的训练与绘图命令

附录 A：CIFAR-10 三模型对比实验（ml-task2-cnn）

CIFAR-10 实验采用可配置 CNN 结构，通过改变通道数、卷积层数和 BatchNorm 设置，构建 base_cnn、wide_cnn、deep_cnn 三种模型。三种模型共享同一训练流程和评价指标，保证对比结果具有可比性。

A.1 三种模型配置

代码（python）

```
PRESET_MODEL_CONFIGS = {
    "base_cnn": {
        "channels": [32, 64, 128],
        "stage_depths": [1, 1, 1],
        "kernel_size": 3,
        "use_batchnorm": False,
        "use_dropout": True,
        "dropout_p": 0.5,
    },
    "wide_cnn": {
        "channels": [64, 128, 256],
        "stage_depths": [1, 1, 1],
        "kernel_size": 3,
        "use_batchnorm": False,
        "use_dropout": True,
        "dropout_p": 0.5,
    },
}
```

```

"deep_cnn": {
    "channels": [32, 64, 128],
    "stage_depths": [2, 2, 2],
    "kernel_size": 3,
    "use_batchnorm": True,
    "use_dropout": True,
    "dropout_p": 0.5,
},
}

```

说明: base_cnn 为基础 CNN 网络; wide_cnn 在保持深度不变的情况下增大通道数量; deep_cnn 增加每个 stage 的卷积层数, 并引入 BatchNorm 层以稳定训练。

A.2 可配置 CNN 模型结构

代码 (python)

```

class ConfigurableCNN(nn.Module):
    def __init__(self, num_classes=10, channels=(32, 64, 128),
                 stage_depths=(1, 1, 1), kernel_size=3,
                 use_batchnorm=False, use_dropout=True, dropout_p=0.5):
        super().__init__()
        padding = kernel_size // 2
        layers = []
        in_channels = 3

        for out_channels, depth in zip(channels, stage_depths):
            for _ in range(depth):
                layers.append(nn.Conv2d(in_channels, out_channels,
                                       kernel_size=kernel_size, padding=padding))

                if use_batchnorm:
                    layers.append(nn.BatchNorm2d(out_channels))
                layers.append(nn.ReLU(inplace=True))
                in_channels = out_channels
            layers.append(nn.MaxPool2d(2))

        self.features = nn.Sequential(*layers)
        self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))

        classifier = [
            nn.Flatten(),
            nn.Linear(channels[-1], 256),
            nn.ReLU(inplace=True),
        ]

        if use_dropout:
            classifier.append(nn.Dropout(dropout_p))

```

```
classifier.append(nn.Linear(256, num_classes))

self.classifier = nn.Sequential(*classifier)

def forward(self, x):
```

说明：该模型通过 `channels` 控制通道数，通过 `stage_depths` 控制每个 `stage` 的卷积层数，并可选择是否启用 `BatchNorm` 与 `Dropout`。最终输出 10 个类别的分类结果。

A.3 数据加载与预处理

代码（python）

```
def get_dataloaders(batch_size=128, num_workers=2):
    transform_train = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                              (0.2023, 0.1994, 0.2010)),
    ])
    transform_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                              (0.2023, 0.1994, 0.2010)),
    ])

    train_set = datasets.CIFAR10(root=cifar_root, train=True,
                                  download=False, transform=transform_train)
    test_set = datasets.CIFAR10(root=cifar_root, train=False,
                                  download=False, transform=transform_test)

    train_loader = DataLoader(train_set, batch_size=batch_size,
                              shuffle=True, num_workers=num_workers)
    test_loader = DataLoader(test_set, batch_size=batch_size,
                              shuffle=False, num_workers=num_workers)
```

说明：训练集使用随机裁剪和随机水平翻转进行数据增强，以提升模型对位置变化和镜像变化的鲁棒性；测试集仅进行标准化处理，避免随机增强影响评价结果。

A.4 单轮训练与评估函数

代码（python）

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()

    total_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
```

```

        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * labels.size(0)
        correct += outputs.max(1)[1].eq(labels).sum().item()
        total += labels.size(0)

    return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)

        total_loss += loss.item() * labels.size(0)
        correct += outputs.max(1)[1].eq(labels).sum().item()
        total += labels.size(0)

```

说明：train_one_epoch 用于训练模型并更新参数，evaluate 用于在测试集上计算损失和准确率。两者都返回平均 loss 与 accuracy，便于后续记录和绘图。

A.5 训练主流程与日志保存

代码（python）

```

model = build_model(args.model).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=20)
for epoch in range(1, 21):
    train_loss, train_acc = train_one_epoch(model, train_loader,
                                             criterion, optimizer, device)

    test_loss, test_acc = evaluate(model, test_loader,
                                   criterion, device)

    scheduler.step()

    log_rows.append({

```

```

        "epoch": epoch,
        "train_loss": train_loss,
        "train_acc": train_acc,
        "test_loss": test_loss,
        "test_acc": test_acc,
    })

    if test_acc > best_test_acc:
        torch.save({
            "model_name": args.model,
            "model_config": model_config,
            "model_state_dict": model.state_dict(),
            "test_acc": test_acc,
        }, f"outputs/models/{args.model}_best.pth")

```

说明：每一轮训练后保存 train_loss、train_acc、test_loss、test_acc 四项指标，并在测试准确率提升时自动保存最佳模型权重文件。

A.6 一键运行实验命令

代码（bash）

```

cd ml-task2-cnn
python run_all_experiments.py --epochs 20 --device auto

```

附录 B：MNIST 手写数字识别实验（mnist_task）

MNIST 实验包含两个部分：第一部分训练 SimpleCNN 与 BetterCNN 两种网络并进行测试集评估；第二部分使用训练后的 BetterCNN 对外部手写数字图片进行预处理和预测。

B.1 两种网络模型定义

代码（python）

```

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),

```

```

        nn.Linear(32 * 7 * 7, 128),
        nn.ReLU(inplace=True),
        nn.Linear(128, 10),
    )

    def forward(self, x):
        return self.classifier(self.features(x))

class BetterCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(p=0.25),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 7 * 7, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(256, 10),
        )

```

说明：SimpleCNN 是基础对照网络，结构较简单；BetterCNN 增加 BatchNorm 与 Dropout，并提升卷积通道数，用于增强模型泛化能力。

B.2 MNIST 数据集加载代码

代码（python）

```

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)),])

train_dataset = datasets.MNIST(root=DATA_DIR, train=True,
                                download=True, transform=transform)

test_dataset = datasets.MNIST(root=DATA_DIR, train=False,

```


说明： MNIST 图像为 $1 \times 28 \times 28$ 的灰度图，使用 ToTensor 转换为张量，并使用 MNIST 常用均值和标准差进行标准化。

B.3 训练循环与日志记录

代码 (python)

```
for epoch in range(1, args.epochs + 1):
    train_loss, train_acc = run_one_epoch(
        model, train_loader, criterion, device, optimizer)
    with torch.no_grad():
        test_loss, test_acc = run_one_epoch(
            model, test_loader, criterion, device)

    history.append({
        "epoch": epoch,
        "train_loss": train_loss,
        "train_acc": train_acc,
        "test_loss": test_loss,
        "test_acc": test_acc,
    })

torch.save({
    "model_name": args.model,
    "model_state_dict": model.state_dict(),
```

说明： 每个 epoch 记录训练集和测试集的 loss 与 accuracy，并将模型权重保存到 outputs/models/ 目录，供后续图片推理调用。

B.4 实拍图片预处理与预测（核心内容）

代码 (python)

```
def preprocess_image(image_path):
    original = Image.open(image_path).convert("RGB")
    gray = ImageOps.grayscale(original)
    gray = ImageOps.autocontrast(gray)

    # 1. 裁剪出手写数字区域
    bbox = find_digit_bbox(gray)
    cropped = gray.crop(bbox)

    # 2. 补齐为正方形图片
    square = make_square(cropped, fill=255)

    # 3. 缩放到 28x28 像素
    resized = square.resize((28, 28), Image.Resampling.LANCZOS)
```

```
# 4. 转换为 MNIST 格式：黑底白字
mnist_like = ImageOps.invert(resized)

# 5. 归一化并转为模型输入张量
tensor = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)),
])(mnist_like).unsqueeze(0)

return tensor
```

代码（python）

```
with torch.no_grad():
    outputs = model(image_tensor.to(device))
    probabilities = torch.softmax(outputs, dim=1)
    confidence, prediction = probabilities.max(dim=1)
print(f"预测结果: {prediction.item()}")
print(f"置信度: {confidence.item():.4f}")
```

说明：预处理流程为“原图→灰度图→裁剪数字区域→补成正方形→缩放至 28×28 →像素反色→归一化→构造模型输入张量”。该流程的目标是使外部手写图片尽量接近 MNIST 的输入分布。

B.5 训练与预测命令

代码（bash）

```
# 模型训练
python src/train.py --model simple_cnn --epochs 10
python src/train.py --model better_cnn --epochs 10

# 绘制训练曲线图
python src/plot_results.py

# 识别自己拍摄的手写数字照片
python src/predict_image.py \
    --image images/digit_7.jpg \
```

附录 C：两个实验的统一项目结构

两个实验的项目结构保持一致，均采用 models.py、train.py、plot_results.py 与 outputs/ 目录组织代码和结果。

模块	功能
models.py	定义神经网络结构
train.py	模型训练、评估，保存日志与权重文件
plot_results.py	绘制损失与准确率变化曲线

outputs/logs/	保存每一轮训练指标 CSV 文件
outputs/models/	保存最优模型权重
outputs/figures/	保存所有实验图表

统一日志字段: train_loss、train_acc、test_loss、test_acc。所有指标可直接用于绘图与结果分析。

附录 D：可直接粘贴到终端的运行命令

D.1 CIFAR-10 实验

代码 (bash)

```
conda activate ai
cd ml-task2-cnn
python run_all_experiments.py --epochs 20 --device auto
python src/plot_results.py --all
```

D.2 MNIST 实验

代码 (bash)

```
conda activate ai
cd mnist_task
python src/train.py --model simple_cnn --epochs 10
python src/train.py --model better_cnn --epochs 10
python src/plot_results.py
```

本附录对应项目路径: 任务二/ml-task2-cnn 与 任务二/mnist_task。